- 在ts代码中，将vue组件转化成html代码
  - 说明：使用情景包括利用vue组件开发leaflet的marker等，凡是可以传html的，都可以用vue组件来做
  - 使用
    - h
      - 将组件渲染成虚拟dom
      - 记得引入组件的prop类型，用来校验参数
    - render
      - 将虚拟dom渲染成真实dom
      - 无文档，可根据ts类型学习使用方式
  - 示例代码

```
import Marker from './Marker/Marker.vue'

const container = document.createElement('div')
render(h(Marker), container)
const icon = L.divIcon({
  html: container,
  className: 'map-marker'
})
const marker = L.marker([lat, lon], { icon })
marker.addTo(layerGroup)
```

  - 相关
    - cloneVNode()、isVNode()
    - resolveComponent()、resolveDirective()
    - withDirectives()、withModifiers()

- 事件重复触发
  - 多个参数触发同一事件，其中一个参数的改变会异步改变另一个参数
    - 侦听会引起其他参数改变的参数，回调函数中关闭侦听剩余参数的侦听器，异步事件结束后开启剩余参数的侦听器（推荐）
      - 示例代码

```
let watchStopHandle: WatchStopHandle | undefined
watch(
  pattern,
  () => {
    watchStopHandle && watchStopHandle()
```

```
    await Promise.all([]) // 异步事件
    createWatcher()
  },
  {
    immediate: true
  }
)
function createWatcher() {
  watchStopHandle = watch(
    [altitudeElementLayerList, currentTime],
    () => {
      // 侦听器逻辑
    },
    {
      immediate: true
    }
  )
}
```

        - 使用onCleanup清除上一次的副作用
    - 对象值未改变，但索引改变了
        - 使用isEqual

- 功能性
    - Teleport
        - SSR环境使用受限，详情参考服务端渲染 (SSR)-Teleports
    - 泛型组件
        - 无法传递泛型，只能使用泛型的类型推导功能
        - 无法获取泛型组件的实例类型（泛型组件的类型，无法通过InstanceType的泛型限制）
    - onRenderTracked()、onRenderTriggered()、响应性调试
        - 示例代码

```
<template>
  <el-input v-model="reaData.name" />
  <el-input v-model="reaData.age" />
  <el-input v-model="refData" />
</template>

<script setup lang="ts">
```

```
const reaData = reactive({
  name: 'abc',
  age: '123'
})
const refData = ref('bbb')

onRenderTracked(event => {
  console.log('onRenderTracked', event)
})

onRenderTriggered(event => {
  console.log('onRenderTriggered', event)
})
</script>
```

    - 说明
      - 组件内、computed、watch、watchEffect都可用
      - 对reactive友好，能根据key区别出是那个数据
      - 对ref不友好，因为key都是value
  - 侦听器的onCleanup
    - watch()、watchEffect()都可用
    - 异步事件前调用，用来清除上一次执行产生的副作用
    - 典型的，用来取消之前的未完成的异步请求
  - watchEffect()
    - 不建议使用，出现问题难以调试
    - 坚持使用的话，建议用onTrigger调试
  - effectScope()、getCurrentScope()、onScopeDispose()
    - 详情参考RFC
  - onErrorCaptured()、app.config.errorHandler

```
// 子组件
throw new Error('我，Children。报错了！')

// 父组件
onErrorCaptured((err, instance, info) => {
  console.log('组件内报错了！', err, instance, info)
  return false
})

```

```
// 入口文件
app.config.errorHandler = (err, instance, info) => {
  console.log('应用报错了！', err, instance, info)
}
```

- provide()、inject()
  - 当子组件多或层级深的时候使用，比props好用
  - 把整个reactive对象或ref传递，以保持响应性
- CSS Modules、CSS中的v-bind()

```
<template>
  <el-input v-model="theme.color" />
  <p>hello</p>
</template>

<script setup>
const theme = reactive({
  color: 'red'
})
</script>

<style scoped>
p {
  color: v-bind('theme.color');
}
</style>
```

- v-pre
- Transition
- 自定义语法块、语法块的src导入
- defineOptions()、defineSlots()

```
defineOptions({
  name: '定义的组件名字'
})

// ------------------------------

```

```
<template>
  <div class="home">
    <slot name="slot1" username="123" />
    <slot name="slot2" :age="123" />
  </div>
</template>

<script setup lang="ts">
defineSlots<{
  slot1(props: { username: string }): any
  slot2(props: { age: number }): any
}>()
</script>
```

- mergeProps()